Project 01: Volume vs. Elite Defenses

Complete walkthrough — what we built, why, and how to explain every piece

This guide explains the entire project end-to-end: the basketball question, the data source, every concept you need to know, every block of code, and what to do next to ship it to GitHub. It is written so you can read it once, understand it, and explain it in an interview.

1. The big picture

The question

How much does a star scorer's points-per-game drop when they play the league's top-5 defenses, compared to their season average against everyone?

Why this matters in basketball

30 PPG against a tanking team is not 30 PPG against the Celtics. Scorers pile up easy buckets against weak defenses, then struggle when the opponent has real personnel, real schemes, and real effort. Coaches and GMs already know this intuitively. Your job as an analyst is to put a clean number on it.

Why it matters for a GM (the contract question)

A GM doesn't really care about regular-season PPG. They care about **playoff PPG**, because every playoff opponent is roughly elite. If you're paying a player $40M per year, you're paying for production when the game gets hard. A scorer whose volume craters against good defenses is essentially padding stats against bad teams — bad value at a max contract.

The story you can tell

“Player X averages 28 PPG over the regular season, but only 21 PPG against the top-5 defenses. Player Y averages 26 PPG, and 25 PPG against elite defenses — a one-point drop. For playoff purposes, Y is the better bet, even though X looks like the louder scorer on paper.”

2. The data — what it is and where it comes from

Source

The data comes from **NBA.com's official stats API** — the same data feed that powers the pages you see at nba.com/stats. We access it through a free, open-source Python library called **nba_api**, which is the analyst community's standard tool for this. It's the same source used by FiveThirtyEight, Cleaning the Glass, and most NBA writers who work with raw numbers.

Season pulled

The script is configured for the **2025-26 NBA regular season** (set on line 23 of `src/analysis.py` via `SEASON = "2025-26"`). Change that one string to pull a different season, e.g. `"2024-25"`.

Two endpoints, two pulls

1. **leaguedashteamstats** — returns one row per team with their advanced stats, including Defensive Rating (DRtg). 30 rows total. This tells us who the elite defenses are.

2. **leaguegamelog** — returns one row per player per game played all season. Around 27,000 rows. Each row has the player's name, points scored, FG attempts, FT attempts, the opponent, and the date.

The script joins these two datasets (every game gets labeled with the opponent's DRtg tier), then aggregates per player. Nothing is uploaded anywhere — the data sits on your hard drive in the `data/` folder.

3. The concepts you need to know

DRtg (Defensive Rating)

DRtg = points allowed per 100 possessions. Lower is better. A team with a DRtg of 108 lets up 108 points for every 100 trips down the floor. A team at 118 lets up 118.

We use season-long team DRtg as our proxy for “defense quality.” The top-5 teams by DRtg are our “elite” bucket. We could have used playoff DRtg or post-trade-deadline DRtg for more precision — that's a future improvement.

PPG (Points Per Game)

Just average points scored per game played. Crude but it's the headline scoring stat everyone knows, so it's what we communicate the result in.

TS% (True Shooting Percentage)

TS% = PTS ÷ (2 × (FGA + 0.44 × FTA)). It's the modern scoring efficiency stat because it accounts for the fact that 3-pointers are worth more than 2-pointers, and free throws matter too. A FG% of 50% with no free throws and no threes is much worse than a FG% of 45% with lots of threes and free throws.

The 0.44 comes from research: roughly 44% of a player's free throw attempts “cost” them a true possession (the others are technicals, and-1s, etc.). We include TS% in the comparison because *how* a player's scoring drops matters: do they still take the same shots and miss, or do they shoot less efficiently?

Opponent tiers

We bucket the 30 teams by their DRtg rank into four groups:

Tier	DRtg rank	What it represents
Elite	1-5	Top defenses, what you face in the playoffs
Good	6-15	Above-average defenses
Average	16-20	Middle of the pack
Poor	21-30	Bottom-third defenses, the teams that pad stats

Why the 10-games minimum filter

Each player has at most ~17 games against the top-5 defenses in a season (each team plays each other 2-4 times). If we don't filter, a guy who happened to drop 40 against one elite team in his one game vs. them looks like a hero. The 10-game floor smooths out small-sample noise.

Why the 20-PPG minimum filter

We only care about *star scorers* — the players who are paid to score and are expected to score in the playoffs. 20 PPG is the standard cutoff for a “number-one scoring option.”

4. The code, block by block

The whole pipeline lives in one file: `src/analysis.py`. Here is what each section does and why. You don't need to memorize the syntax — you need to be able to explain *what each block does*.

Block 1 — Imports

```
import os
import time
import pandas as pd
import matplotlib.pyplot as plt
from nba_api.stats.endpoints import leaguegamelog, leaguedashteamstats
```

os handles file paths (so the script works on any computer). **time** lets us pause briefly between API calls to be polite to NBA's server. **pandas** is the standard Python library for working with tables of data (dataframes). **matplotlib** is the standard Python library for making charts. **nba_api** is the library that talks to NBA.com's stats API. We import the two endpoints we'll use.

Block 2 — Config

```
SEASON = "2025-26"
MIN_PPG_OVERALL = 20.0
MIN_GAMES_VS_ELITE = 10
TOP_N_FOR_CHART = 15
```

All the knobs in one place. **SEASON** picks which year. **MIN_PPG_OVERALL** is the “star scorer” threshold. **MIN_GAMES_VS_ELITE** kills small-sample noise. **TOP_N_FOR_CHART** says how many players make it onto the slope chart. Putting these at the top is good practice — anyone reading the code can tune the analysis without hunting through the body.

Block 3 — File paths

```
HERE = os.path.dirname(os.path.abspath(__file__))
PROJECT_DIR = os.path.dirname(HERE)
DATA_DIR = os.path.join(PROJECT_DIR, "data")
FIG_DIR = os.path.join(PROJECT_DIR, "figures")
os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(FIG_DIR, exist_ok=True)
```

Figures out where the script lives (HERE), then assumes the project folder is one level up. Builds the paths to `data/` and `figures/`, and creates them if they don't exist. This makes the script portable — it works no matter where you put the repo on your computer.

Block 4 — fetch_team_drtg()

```
df = leaguedashteamstats.LeagueDashTeamStats(
season=season,
measure_type_detailed_defense="Advanced",
per_mode_detailed="PerGame",
).get_data_frames()[0]
```

Calls NBA.com's API for the “advanced team stats” table. Returns a dataframe with one row per team and many columns; we only need `TEAM_ID`, `TEAM_NAME`, and `DEF_RATING`.

```
drtg["DRTG_RANK"] = drtg["DEF_RATING"].rank(method="min").astype(int)
drtg["TIER"] = drtg["DRTG_RANK"].apply(tier)
```

Ranks the 30 teams by DEF_RATING (best defense = rank 1), then assigns each team to a tier (Elite, Good, Average, Poor) using the nested `tier()` function. Saves the result to `data/team_drtg.csv`.

Block 5 — fetch_game_logs()

```
df = leaguegamelog.LeagueGameLog(
season=season,
player_or_team_abbreviation="P",
).get_data_frames()[0]
```

Calls NBA.com's API for every player-game in the season. The `"P"` argument means “return one row per *player* per game,” not one row per *team* per game. Around 27,000 rows. Each row has the player ID, name, points, FGA, FTA, and the `MATCHUP` string like "LAL @ BOS".

Block 6 — attach_opponent_tier()

This is the most important block. We do the join here.

```
abbr_to_id = dict(zip(logs["TEAM_ABBREVIATION"], logs["TEAM_ID"]))
logs["OPP_ABBR"] = logs["MATCHUP"].str.split().str[-1]
logs["OPP_TEAM_ID"] = logs["OPP_ABBR"].map(abbr_to_id)
```

The MATCHUP column looks like "LAL vs. BOS" or "LAL @ BOS". The last word is always the opponent's three-letter abbreviation. We extract it, then look up its team ID using the abbreviation-to-ID map built from the logs themselves.

```
joined = logs.merge(
drtg[["TEAM_ID", "TEAM_NAME", "DRTG_RANK", "TIER"]].rename(
columns={"TEAM_ID": "OPP_TEAM_ID", ...}
),
on="OPP_TEAM_ID", how="left",
)
```

Joins the game logs with the team DRtg table on the opponent team ID. After this line, every single player-game row knows which tier the opponent belongs to.

```
joined["TS_DENOM"] = 2 * (joined["FGA"] + 0.44 * joined["FTA"])
joined["TS_PCT"] = joined["PTS"] / joined["TS_DENOM"].replace(0, pd.NA)
```

Computes True Shooting Percentage per game. We replace zeros in the denominator with NA so we don't get divide-by-zero errors for players who didn't shoot at all.

Block 7 — build_comparison()

```
overall = games.groupby(["PLAYER_ID", "PLAYER_NAME"]).agg(
GAMES_ALL=("GAME_ID", "count"),
PPG_ALL=("PTS", "mean"),
...
).reset_index()
```

Groups all player-game rows by player and computes their season totals: games played, average points, total points, total TS denominator. Standard groupby-aggregate pattern in pandas.

```
elite = games[games["OPP_TIER"] == "Elite (1-5)"].groupby(...).agg(...)
```

Same thing, but only for games where the opponent was a top-5 defense. Filter first, then aggregate.

```
comp = overall.merge(elite, on=["PLAYER_ID", "PLAYER_NAME"], how="inner")
comp["PPG_DROP"] = comp["PPG_ALL"] - comp["PPG_ELITE"]
comp["PCT_DROP"] = (comp["PPG_DROP"] / comp["PPG_ALL"]) * 100
```

Merges the two aggregates so each player gets one row with both their overall stats and their vs-elite stats side by side. Computes the absolute and percentage drop.

```
stars = comp[
(comp["PPG_ALL"] >= MIN_PPG_OVERALL)
& (comp["GAMES_ELITE"] >= MIN_GAMES_VS_ELITE)
].sort_values("PPG_ALL", ascending=False)
```

Filters to the qualifying star scorers (20+ PPG with 10+ games vs elite defenses), sorts by overall scoring. Saves to `data/comparison.csv`.

Block 8 — build_slope_chart()

Loops through the top 15 players and draws one line per player from their PPG_ALL on the left to their PPG_ELITE on the right. Lines are colored red if the player drops 3+ PPG (the “regular season hero”), blue otherwise (the “any-opponent scorer”). The annotation next to each line shows the player's name and the two numbers. The chart is saved to `figures/headline.png` at 160 dpi.

Block 9 — main()

Orchestrates everything in order: pull DRtg → pull game logs → join → aggregate → chart. Prints the top-10 biggest droppers and top-10 most consistent scorers to the terminal so you can see the headline result without opening any CSV.

5. What's in your project folder now

File	What's in it
`README.md`	GitHub-facing project page; embeds the headline chart
`WALKTHROUGH.md`	Step-by-step click-by-click guide (paired with this PDF)
`writeup.md`	200-word write-up template you fill in
`requirements.txt`	Package list, so anyone can reproduce your setup
`src/analysis.py`	The entire pipeline in one file
`data/team_drtg.csv`	30 teams, ranked by defense, with tier labels
`data/game_logs_joined.csv`	Every player-game with opponent tier attached
`data/comparison.csv`	The answer table — star scorers, PPG drop, TS%
`figures/headline.png`	The slope chart for the README

6. What to do next — click by click

Step 1 — Look at team_drtg.csv to verify the data

In VS Code, expand `data/` in the left sidebar. Click `team_drtg.csv`. Scan the team names — they should all be real NBA teams. The team at `DRTG_RANK = 1` is the league's best defense for 2025-26. Write down its name; you'll mention it in your write-up.

Step 2 — Look at comparison.csv to get your headline numbers

Open `data/comparison.csv`. You'll see one row per qualifying star scorer. The columns that matter most:

- **PPG_ALL** — their overall season points per game
- **PPG_ELITE** — their PPG when playing a top-5 defense
- **PPG_DROP** — the difference (positive = scored less vs. elite)
- **PCT_DROP** — the drop as a percentage of their overall scoring

Find the player with the biggest `PPG_DROP` (the regular-season hero) and the player with the smallest or negative drop (the any-opponent scorer). Write down both names and their numbers.

Step 3 — Update README.md

Open `README.md`. Line 5 says *(to fill in once you finish the analysis)*. Replace it with a sentence using the numbers you just wrote down. Save with Ctrl+S.

Step 4 — Fill in writeup.md

Open `writeup.md`. It has blanks marked with three underscores (___). Fill them in. Target ~200 words total. Save with Ctrl+S.

Step 5 — Commit and push to GitHub

In the VS Code terminal (make sure `(.venv)` is showing), make sure you're at the repo root:

```
cd C:\Users\User\Documents\Claude\Projects\ANALYTICS PORTFOLIO\NBA-Analytics-Portfolio
```

Then run these three commands one at a time:

```
git add .
git commit -m "Project 01: opponent-adjusted scoring analysis"
git push
```

The first push will ask you to authenticate with GitHub. Easiest path: install GitHub CLI from cli.github.com, run `gh auth login` once, follow the browser prompts. After that, every `git push` just works.

Step 6 — Verify on GitHub

Refresh your repo page in the browser. Navigate to `projects/01-volume-vs-elite-defenses/README.md`. The headline chart should now render inline. That's your shareable proof-of-work.

7. How to talk about this in an interview

The 30-second elevator pitch

“I wanted to quantify how much star scorers' production drops against elite defenses, because that's the lens a GM actually cares about when paying for playoff value. I pulled every player-game from the 2025-26 NBA season via the official stats API, joined each game to the opponent's season defensive rating, bucketed opponents into four tiers, and computed each star scorer's PPG and true-shooting against top-5 defenses versus everyone. The headline chart shows who holds up and who doesn't.”

Likely follow-up questions and good answers

Q: Why DRtg and not Defensive Rating from a specific period?

Honest answer: season-long DRtg is the simplest available signal. Better versions would use post-trade-deadline DRtg (rosters change), or DRtg adjusted for the lineup that was on the floor for each specific game. I went with the simple version first to get a clean baseline.

Q: Why points per game and not points per 100 possessions?

Good answer: PPG is what people communicate scoring in, so I led with it. Per-100 would control for pace differences, which is a real concern — elite defenses tend to play slower, which suppresses PPG even before they affect efficiency. Including TS% in the comparison partially addresses this; a full version would add per-100 scoring as a third column.

Q: Sample size — 10 games is small, isn't it?

Yes. 10-17 games against elite defenses is a real limitation. To get larger samples I'd pool 2-3 seasons. The 10-game floor is a tradeoff between including enough players and trusting the per-player number.

Q: How would you extend this?

Several ways:
• Multi-season pool to grow the per-player sample
• Adjust for home/away and back-to-backs
• Compare regular-season drop vs. actual playoff PPG to test whether the metric predicts playoff performance
• Add usage rate to see whether the scorer's role changes vs. elite defenses (taking fewer shots vs. shooting worse)

Q: What surprised you?

Answer with a specific name from *your* data — the biggest dropper or the most consistent scorer. That's where having opened `comparison.csv` matters: you have an actual fact at the ready.

Honest limitations to volunteer

• Season-long DRtg ignores in-season changes (trades, injuries to defenders)
• PPG is volume; doesn't separate quantity from efficiency
• The 10-game minimum drops some interesting players (e.g., late-season trades)
• No pace adjustment, no home/away split, no minutes-played control

Volunteering these honestly is what separates an analyst from someone repeating a buzzword. Always state them.

Closing

This is Project 01 of a portfolio. Once you push it, the structure to copy for Project 02 is: `projects/02-something/` with `src/`, `data/`, `figures/`, `README.md`, `writeup.md`. The pipeline pattern (pull → join → aggregate → visualize → write up) repeats.